

Local Kubernetes Docker Registry Guide

Rancher Desktop + Kubernetes (containerd)

A clean, practical lab guide for deploying a Docker Registry inside a local Kubernetes cluster, exposing it on your workstation, and manually copying an image from a GitLab Container Registry into it. Includes the real blockers we hit (ImagePullBackOff, Docker CLI vs containerd) and the concepts that caused confusion (tags, “latest”, PVC vs emptyDir, local registry vs local containerd, and what a “rollout” means).

Why this matters: when Rancher Desktop uses containerd, you use nerdctl. The Docker CLI talks to a Docker daemon (dockerd), which you are not running in this mode.

Container Engine



containerd

Namespaces for container images; use with nerdctl.



dockerd (moby)

Docker API; use with Docker CLI.

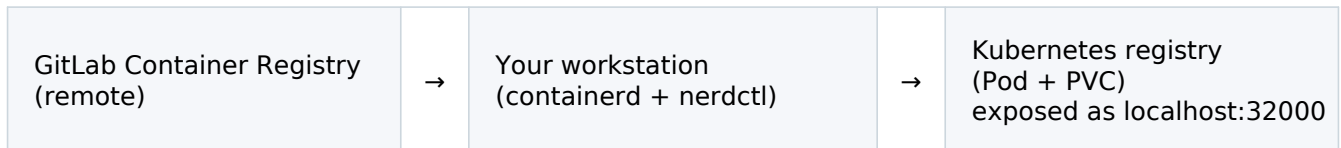
1. What you're building

Goal: Run a private Docker Registry (registry v2) inside your local Kubernetes cluster and make it reachable from Windows at `http://localhost:32000`.

Acceptance criteria: (1) Registry installed and ready. (2) Latest PetClinic image present in the registry (copied manually).

Out of scope: Automation (CronJobs), TLS/HTTPS, multi-replica HA registry, and PremDava-specific networking.

2. Architecture at a glance



Key idea: a registry is a server that stores images (like a mini Docker Hub). containerd is the runtime that runs containers and keeps a local image cache. You copy images between registries with push/pull.

3. Prerequisites

You need:

- Rancher Desktop with Kubernetes enabled
- Container Engine = containerd
- kubectl configured for the Rancher Desktop cluster
- nerdctl installed/available (usually bundled with Rancher Desktop)
- Network access to your GitLab registry

3.1 Which CLI should you use? (docker vs nerdctl vs podman)

In this setup you are on containerd, so nerdctl is the right tool for pulling/tagging/pushing images.

If you ever switch Rancher Desktop to dockerd (moby), then the standard docker CLI becomes the right tool.

Podman is fine too, but only if it is connected to the same runtime you expect your tools to use (your WSL Podman machine is separate from Rancher Desktop).

Useful “what am I using?” checks:

```
docker context ls
# If Rancher Desktop runs containerd, docker may still point to Docker Desktop
and fail.
nerdctl version
kubectl cluster-info
```

4. Kubernetes manifests (what each file does)

You applied four YAML files. Here is what each one creates and why it exists.

4.1 00-namespace.yaml

Creates a dedicated namespace called registry. A namespace is a logical folder for resources, making cleanup and troubleshooting easier.

4.2 01-pvc.yaml

Creates a PersistentVolumeClaim (PVC) (10Gi). A PVC is disk storage that remains even if the registry Pod is replaced. The registry stores image layers and manifests under /var/lib/registry, so mounting a PVC there makes image storage persistent.

PVC vs emptyDir (what you lose with emptyDir)

- emptyDir is temporary Pod storage: if the Pod is deleted/recreated, the registry becomes empty.
- PVC survives Pod replacement: images remain after restarts/rollouts.

In your lab, using emptyDir only risks losing the local registry contents (repos/tags/layers). Nothing in GitLab is affected; you would just push again.

4.3 02-registry-deployment.yaml

Creates a Deployment that runs the registry container and keeps it running over time. A Deployment manages Pods (restarts, replacements, updates).

Key points: replicas: 1, registry listens on 5000, and the PVC is mounted into /var/lib/registry.

4.4 03-registry-service.yaml

Creates a Service (type NodePort) to expose the registry outside the cluster on localhost:32000.

Think of it as routing: localhost:32000 → Service → Pod → container port 5000.

5. Deploy and validate (commands + meaning)

5.1 Apply the manifests

“Apply” means: create resources if missing, or update them if they already exist.

```
kubectl apply -f 00-namespace.yaml
kubectl apply -f 01-pvc.yaml
kubectl apply -f 02-registry-deployment.yaml
kubectl apply -f 03-registry-service.yaml
```

5.2 Confirm the Pod is running

A registry is usable only when the Pod is Running and Ready 1/1.

```
kubectl -n registry get pod -o wide
```

5.3 Confirm storage is bound

PVC should show Bound. If it's Pending, the cluster couldn't provision storage.

```
kubectl -n registry get pvc
kubectl get storageclass
```

5.4 Confirm the registry responds

Registry v2 exposes an API under /v2/. Healthy output looks like HTTP 200 and {}.

```
curl -v http://localhost:32000/v2/
```

Why localhost:5000 failed for you

Port 5000 is the registry's internal port. Your host can only reach 5000 if you create a temporary tunnel with kubectl port-forward. Without port-forward, nothing listens on localhost:5000.

```
kubectl -n registry port-forward svc/docker-registry 5000:5000  
# then: curl -v http://localhost:5000/v2/
```

6. Blockers we hit (and the fixes)

6.1 ImagePullBackOff (registry Pod would not start)

Meaning: Kubernetes could not download the container image for the Pod. The Service may exist, but there is no working backend Pod.

Common reasons: Docker Hub blocked/rate-limited; corporate proxy; DNS/egress issues.

Fix: Use a registry image hosted somewhere reachable (e.g., Quay) and restart the Deployment.

```
kubectl -n registry set image deploy/docker-registry  
registry=quay.io/libpod/registry:2  
kubectl -n registry rollout status deploy/docker-registry
```

6.2 "Empty reply from server" when curling localhost:32000

Usually means the NodePort is reachable but there is no ready Pod behind the Service (often because of ImagePullBackOff).

Check endpoints: if the endpoints list is empty, the Service has nothing to route to.

```
kubectl -n registry get pod  
kubectl -n registry get endpoints docker-registry
```

6.3 Docker CLI error about dockerDesktopLinuxEngine

You were in the desktop-linux Docker context (Docker Desktop), but you were not running Docker Desktop. In addition, your Rancher Desktop engine is containerd, so the Docker CLI has no daemon to talk to.

Fix: Use nerdctl with containerd. Alternative: switch Rancher Desktop engine to dockerd (moby) if you want to use Docker CLI.

7. Manual image copy (GitLab → local registry)

Core flow: login → pull → tag → push. Use placeholders; never paste real secrets into docs.

7.1 Safe example variables (no real credentials)

```
$GITLAB_REGISTRY="<gitlab-registry-host>"  
$GITLAB_USER="<your-username-or-email>"
```

```
$GITLAB_TOKEN="<credential-with-read-registry>"
$GITLAB_IMAGE="<gitlab-registry-host>/<group>/<project>/<image>"
$TAG="latest"
$LOCAL_IMAGE="localhost:32000/<repo-name>:$TAG"
```

7.2 Commands (and what each does)

1) Authenticate to GitLab (enables pulling private images):

```
nerdctl login $GITLAB_REGISTRY -u $GITLAB_USER -p $GITLAB_TOKEN
```

2) Pull the image from GitLab into your local containerd image store:

```
nerdctl pull "$GITLAB_IMAGE:$TAG"
```

3) Create a new name for the same local image, pointing at your local registry:

```
nerdctl tag "$GITLAB_IMAGE:$TAG" $LOCAL_IMAGE
```

4) Push to the local registry. Because your local registry is HTTP, add the insecure flag:

```
nerdctl --insecure-registry push $LOCAL_IMAGE
```

Why tagging is needed

The registry location is part of the image name. Retagging changes the destination from GitLab to localhost:32000 without rebuilding the image.

8. Verification (meets acceptance criteria)

8.1 Registry is ready

Keep this as your basic health check:

```
curl -v http://localhost:32000/v2/
```

8.2 Image is present

Use the registry API to list repositories and tags:

```
curl http://localhost:32000/v2/_catalog
curl http://localhost:32000/v2/<repo-name>/tags/list
```

8.3 Prove persistence (PVC)

Restart the registry Pod and re-check the catalog. If images remain, the PVC is working.

```
kubectl -n registry rollout restart deploy/docker-registry
kubectl -n registry get pod -w
curl http://localhost:32000/v2/_catalog
```

9. Concepts explained (with examples)

9.1 Tags and “latest”

An image reference looks like: host:port/repo:tag. A tag is just a label. “latest” is not special; it only means “the tag you keep updating”.

Example tags you might use: latest, 1.2.3, git-7f3a9c1, 2026-02-17-1450.

Best practice: push both :latest (convenient) and an immutable tag (commit SHA) so you can roll back reliably.

9.2 Local registry vs local containerd

Local registry (localhost:32000) stores images on disk and serves them over HTTP. Local containerd stores images locally for running containers.

Pull = download from a registry into containerd.

Push = upload from containerd into a registry.

9.3 What “rollout” means

A rollout is a managed restart / rolling replacement of Pods in a Deployment. Kubernetes creates a new Pod, waits for it to be ready, then removes the old Pod.

You used rollout restart to: (a) re-trigger image pulling after a fix, and (b) test that your registry data survives Pod replacement (PVC proof).

9.4 NodePort vs port-forward

NodePort is a stable external port (32000) opened by Kubernetes.

port-forward is temporary and exists only while the command is running (e.g., forwarding to 5000).

10. Minimal troubleshooting checklist

If something stops working, run these in order:

```
kubectl -n registry get pod,svc,pvc
kubectl -n registry logs deploy/docker-registry --tail=50
kubectl -n registry describe pod -l app=docker-registry
curl -v http://localhost:32000/v2/
```